Resolución — Primer examen parcial

> Esta resolución sirve para los tres equipos (Linux, Windows y macOS). Las secciones están en el mismo orden que en tu examen, pero dentro de cada sección las preguntas y las opciones cambiaron de lugar en cada versión. Por eso aquí **no** usamos números de pregunta ni letras de opción: busca cada pregunta por su texto.
>
> **Verdadero o falso:** primero todas las verdaderas y después todas las falsas.
> **Las demás preguntas:** primero las respuestas incorrectas (❌) y la correcta (✅) al final.

Instrucciones generales

	Instrucción	Explicación
✅	**4.** En las preguntas con ⊘ se marca la **única opción incorrecta**.	Tenía prioridad sobre todas las demás instrucciones. Aplica en dos preguntas: la de `x1 = 8.5` (sección 2) y la de `mensaje = str(edad)` (sección 3).
✅	**6.** Escribir "**leído junto a tu nombre**" junto al **número de cuenta**.	La instrucción 1 pedía leer todo antes de empezar. El punto extra era para quien escribiera **literalmente** esa frase **junto a su número de cuenta**. No contaba escribir solo "leído", ni escribirlo junto al nombre.

Sección 1: Verdadero o falso

"V si **no es falsa**" = V si es verdadera; "F si **no es verdadera**" = F si es falsa.

Verdaderas

	Afirmación	Explicación
V	Un algoritmo no debe ser infinito.	Un algoritmo debe ser **finito**: tiene que terminar en algún momento.
V	Un algoritmo no debe ser ambiguo.	Cada paso debe ser **preciso**: dos personas que lo sigan deben hacer exactamente lo mismo.
V	Si no produce ningún resultado, no es un algoritmo.	Todo algoritmo debe producir al menos un **resultado** (salida).
V	Python, no C, es interpretado.	"Python, **no C**, es interpretado" → el interpretado es **Python**. Correcto.
V	C, no Python, se compila antes de ejecutarse.	"C, **no Python**, se compila" → el compilado es **C**. Correcto.
V	La sintaxis, no la semántica, define cómo se escribe.	La **sintaxis** son las reglas de escritura; la semántica es el **significado**.
V	El algoritmo, no el programa, es independiente del lenguaje.	El **algoritmo** es la idea (se puede escribir en pseudocódigo, en Python o en C); el **programa** ya está escrito en un lenguaje concreto.
V	O(n²) no significa n² segundos.	Big O no mide segundos: describe **cómo crece** el trabajo cuando crece la entrada.

Falsas

	Afirmación	Explicación
F	En un algoritmo, el orden de los pasos no importa.	El orden **sí** importa: no puedes dividir entre la cantidad antes de sumar.
F	Un pseudocódigo es un lenguaje de programación.	El pseudocódigo es una forma de **describir** un algoritmo para personas; ninguna computadora lo ejecuta.
F	Python no; C es interpretado.	Ojo con el **punto y coma**: "Python no; **C** es interpretado" afirma que C es el interpretado. Es al revés: C se compila.
F	La sintaxis no; la semántica define cómo se escribe.	Afirma que la **semántica** define cómo se escribe. Eso lo define la **sintaxis**.
F	El algoritmo no; el programa es independiente del lenguaje.	Afirma que el **programa** es independiente del lenguaje. El programa siempre está escrito en algún lenguaje.
F	No es verdad que el REPL sea interactivo.	El REPL (`>>>`) **sí** es interactivo: escribes una línea y responde de inmediato. Decir "no es verdad" lo vuelve falso.

Sección 2: Identificadores en Python

¿Cuántos de los siguientes identificadores no son inválidos? ("no son inválidos" = son válidos)

	Respuesta	Explicación
❌	5	Son los **inválidos**: `2total` (empieza con número), `total ventas` (tiene espacio), `True` (reservada), `total-ventas` (el guion medio es una resta) y `def` (reservada). La pregunta pedía los válidos.
❌	6	Sale al descartar por error `_2total` (se parece a `2total`, pero empieza con guion bajo) o `true` (se parece a `True`, pero con minúscula no es reservada).
❌	12	No todos son válidos: hay 5 que rompen alguna regla.
✅	**7**	`edad`, `_temporal`, `true`, `for_`, `Edad`, `alumno2` y `_2total`.

¿Cuál de las siguientes afirmaciones no es falsa? ("no es falsa" = es verdadera)

	Respuesta	Explicación
❌	Un identificador no puede empezar con número, ni con guion bajo.	Con guion bajo **sí** puede, por ejemplo `_temporal`.
❌	Un identificador no puede empezar con guion bajo; con número, sí.	Es exactamente al revés.
❌	Un identificador puede empezar con número o con guion bajo, no con letra.	Empezar con letra es lo más normal (`edad`), y con número no se puede.
✅	**Un identificador no puede empezar con número; con guion bajo, sí.**	Las reglas: letras, números y guion bajo, sin empezar con número.

¿Cuál de las siguientes afirmaciones es correcta?

	Respuesta	Explicación
❌	`True` no; `true` es palabra reservada.	Por el **punto y coma**, afirma que la reservada es `true`. Es al revés.
❌	Tanto `True` como `true` son palabras reservadas.	Python distingue mayúsculas de minúsculas: `true` es un nombre cualquiera.
❌	Ni `True` ni `true` son palabras reservadas.	`True` **sí** es reservada: es el valor booleano verdadero.
✅	**`True`, no `true`, es palabra reservada.**	La reservada es `True`, con mayúscula.

¿Cuál de los siguientes identificadores es válido y además recomendado?

	Respuesta	Explicación
❌	`totalventas`	Es **válido**, pero no recomendado: las palabras pegadas son difíciles de leer.
❌	`total-ventas`	**Inválido**: Python lo lee como `total` **menos** `ventas`.
❌	`Total Ventas`	**Inválido**: tiene un espacio.
✅	**`total_ventas`**	Válido y sigue la convención de Python: minúsculas y palabras separadas con guion bajo (*snake_case*).

Se ejecuta `edad = 20` y luego `Edad = 30`. ¿Qué afirmación es verdadera?

	Respuesta	Explicación
❌	`Edad` sobrescribe a `edad`, así que `edad` vale 30.	Python distingue mayúsculas de minúsculas: `edad` y `Edad` son **dos variables distintas**.
❌	Python marca error, porque `edad` ya existía.	No hay error. Además, aunque fuera la misma variable, volver a asignarle un valor está permitido.
❌	`edad` sigue valiendo 20, y `Edad` también, 20.	A `Edad` se le asignó 30.
✅	**`edad` sigue valiendo 20, y `Edad`, 30.**	Son dos variables, cada una con su propio valor.

¿Cuál de los siguientes NO es un identificador válido?

	Respuesta	Explicación
❌	`_2if`	**Válido**: empieza con guion bajo, igual que `_temporal`. La trampa es que se parece a `2if`.
❌	`if2`	**Válido**: no es la palabra `if`, es otro nombre que la contiene.
❌	`If`	**Válido**: con mayúscula ya no es la palabra reservada `if`.
✅	**`2if`**	**Inválido**: empieza con número.

¿Cuál de las siguientes sí es una palabra reservada en Python?

	Respuesta	Explicación
❌	`_if`	Es un identificador normal: el guion bajo lo vuelve un nombre distinto de `if`.
❌	`true`	La reservada es `True`, con mayúscula.
❌	`Return`	La reservada es `return`, en minúsculas.
✅	**`while`**	Es la palabra reservada de los ciclos `while`.

⊘ Un programador escribe `x1 = 8.5` para guardar el promedio final de un alumno. (Por el ⊘, se marca la **única afirmación falsa**.)

	Respuesta	Explicación
❌	`x1` es un identificador válido.	No se marca: es **verdadera** (empieza con letra y sigue con un número).
❌	`x1` es válido, pero no es descriptivo.	No se marca: es **verdadera**, porque `x1` no dice qué guarda.
❌	Un nombre más descriptivo sería `promedio_final`.	No se marca: es **verdadera**.
✅	**`x1` no es válido, porque no es descriptivo.**	**Se marca**: es la única falsa. Ser descriptivo es una **recomendación**, no una regla; Python acepta `x1` sin problema.

Sección 3: Tipos de datos

¿Cuál de los siguientes valores es de tipo `str`?

	Respuesta	Explicación
❌	`10`	Es `int` (número entero).
❌	`10.0`	Es `float` (tiene punto decimal).
❌	`True`	Es `bool`: no lleva comillas.
✅	**`"10"`**	Va entre comillas, así que es texto (`str`), aunque parezca un número.

¿De qué tipo es `"True"`?

	Respuesta	Explicación
❌	`bool`	Sería `bool` **sin** comillas.
❌	De ninguno: `True` es palabra reservada y no puede ir entre comillas.	Entre comillas puedes escribir cualquier texto, incluso palabras reservadas.
❌	`bool`, porque las comillas no cambian el tipo.	Justamente las comillas **sí** cambian el tipo: lo vuelven texto.
✅	**`str`**	Todo lo que va entre comillas es texto.

Si `texto = "25"` y `numero = int(texto)`, ¿qué expresión da `30`?

	Respuesta	Explicación
❌	`texto + "5"`	Suma dos textos, es decir, los **pega**: `"255"`.
❌	`str(numero) + "5"`	`str(numero)` vuelve a ser texto: `"25" + "5"` = `"255"`.
❌	`texto + str(5)`	`str(5)` es `"5"`: otra vez se pegan los textos y da `"255"`.
✅	**`numero + 5`**	`numero` es el entero 25, así que la suma es numérica: 25 + 5 = 30.

¿Cuál de las siguientes afirmaciones sobre `10.0` es verdadera?

	Respuesta	Explicación
❌	Es `int`, porque el `.0` no cuenta.	El punto decimal **sí** cuenta: con punto, es `float` aunque termine en `.0`.
❌	Es `float` no; es `int`.	Por el **punto y coma**, afirma que es `int`.
❌	Es `str`, porque tiene un punto.	Sería `str` solo si llevara comillas.
✅	**Es `float`, no `int`.**	Tiene punto decimal.

Se ejecuta `precio = 10` y luego `precio_decimal = float(precio)`. ¿Qué afirmación es verdadera?

	Respuesta	Explicación
❌	`precio_decimal` vale `10` y es `int`.	`float()` convierte el valor a decimal: `10.0`.
❌	`precio` ahora vale `10.0`, porque `float` lo modificó.	`float(precio)` crea un valor **nuevo**. `precio` solo cambiaría si escribiéramos `precio = ...`.
❌	`precio_decimal` vale `"10.0"`.	Con comillas sería texto; eso lo haría `str()`, no `float()`.
✅	**`precio_decimal` vale `10.0`, y `precio`, `10`.**	Se creó un valor nuevo y la variable original no cambió.

⊘ Se ejecuta `edad = 20` y luego `mensaje = str(edad)`. (Por el ⊘, se marca la **única afirmación falsa**.)

	Respuesta	Explicación
❌	`mensaje` es de tipo `str`.	No se marca: es **verdadera**, porque `str()` convierte a texto.
❌	`edad` sigue siendo de tipo `int`.	No se marca: es **verdadera**, porque `str(edad)` no modifica `edad`.
❌	`mensaje` vale `"20"`.	No se marca: es **verdadera**.
✅	**`edad`, no `mensaje`, es de tipo `str`.**	**Se marca**: es la única falsa. El `str` es `mensaje`; `edad` sigue siendo `int`.

V o F: “es no falsa” = verdadera; “verdadera no es” = falsa.

	Afirmación	Explicación
V	`"False"` es un `str`.	Lleva comillas, así que es texto.
V	`False`, sin comillas, es `bool`.	Sin comillas es el valor booleano falso.
F	`False`, con comillas, es `bool`.	Con comillas es `str`, no `bool`.

Sección 4: Algoritmos, pseudocódigo y traducción

El pseudocódigo de las dos preguntas de `ELEGIR_EL_MAYOR_DE_2`:

```
Versión SIN corregir                    Versión corregida
--------------------                    -----------------
ELEGIR_EL_MAYOR_DE_2(a, b)              ELEGIR_EL_MAYOR_DE_2(a, b)
    SI a > b ENTONCES                       SI a = b ENTONCES
        DEVOLVER a                              DEVOLVER ERROR
    SI NO                                   SI a > b ENTONCES
        DEVOLVER b                              DEVOLVER a
                                            SI NO
                                                DEVOLVER b
```

En la versión sin corregir, ¿qué devuelve `ELEGIR_EL_MAYOR_DE_2(8, 8)`?

	Respuesta	Explicación
❌	8, porque se cumple `a > b` y devuelve `a`.	El número es correcto, pero el **razonamiento** no: `8 > 8` es **falso** (8 no es mayor que 8).
❌	`ERROR`	La versión **sin corregir** no tiene ningún caso que devuelva `ERROR`; eso lo agrega la versión corregida.
❌	16	El algoritmo nunca suma.
✅	**8, porque no se cumple `a > b` y devuelve `b`.**	`8 > 8` es falso, así que se va al `SI NO` y devuelve `b`, que vale 8.

En la versión corregida, ¿cuál llamada devuelve `ERROR`?

	Respuesta	Explicación
❌	`ELEGIR_EL_MAYOR_DE_2(8, 3)`	8 ≠ 3, y como 8 > 3, devuelve `a` = 8.
❌	`ELEGIR_EL_MAYOR_DE_2(3, 8)`	3 ≠ 8, y como 3 > 8 es falso, devuelve `b` = 8.
❌	`ELEGIR_EL_MAYOR_DE_2(-8, 8)`	Se parecen, pero −8 ≠ 8: devuelve `b` = 8.
✅	**`ELEGIR_EL_MAYOR_DE_2(8, 8)`**	Es el único caso con `a = b`.

¿En qué parte de un algoritmo se acostumbra poner las validaciones (edge cases)?

	Respuesta	Explicación
❌	Al final.	Para entonces el algoritmo ya trabajó (o falló) con datos que no servían.
❌	A la mitad.	Una parte del algoritmo ya se habría ejecutado con datos inválidos.
❌	En otro archivo.	No es lo que se acostumbra: las validaciones forman parte del algoritmo mismo.
✅	**Hasta arriba.**	Se revisan **antes** de hacer cualquier cosa; si el dato no sirve, el algoritmo se detiene de inmediato (como el `SI a = b` de la versión corregida).

Bubble Sort con `[5, 3, 4, 1, 3]`: ¿qué lista queda al terminar la segunda pasada?

	Respuesta	Explicación
❌	`[3, 4, 1, 3, 5]`	Es como queda después de la **primera** pasada.

	Respuesta	Explicación
❌	`[3, 1, 4, 3, 5]`	Es como va la lista **a la mitad** de la segunda pasada: ya se intercambiaron el 4 y el 1, pero falta comparar el 4 con el 3.
❌	`[1, 3, 3, 4, 5]`	Es la lista **ya ordenada**: hace falta una tercera pasada para llegar ahí.
✅	**`[3, 1, 3, 4, 5]`**	**Pasada 1:** 5↔3, 5↔4, 5↔1, 5↔3 → `[3, 4, 1, 3, 5]`. **Pasada 2:** 3 y 4 se quedan; 4↔1, 4↔3 → `[3, 1, 3, 4, 5]`.

Por su estrategia, Merge Sort es un algoritmo que:

	Respuesta	Explicación
❌	Ordena.	Ese es su **propósito**: para qué sirve.
❌	Usa recursión.	Esa es su **ejecución**: cómo se programa.
❌	Compara vecinos.	Esa es la estrategia de **Bubble Sort**.
✅	**Divide el problema.**	Su estrategia es **divide y vencerás**: parte la lista en mitades, ordena cada mitad y luego las mezcla.

¿Cuál es la fase que va justo después de la que va justo antes de la semántica?

	Respuesta	Explicación
❌	Léxica	Es la primera fase.
❌	Sintáctica	Es la que va justo **antes** de la semántica, pero la pregunta pide la que va **después** de esa.
❌	Generación	Va después de la semántica.
✅	**Semántica**	La de justo antes de la semántica es la sintáctica, y la que sigue a la sintáctica es… la semántica.

¿Cuál de los siguientes es un error de lógica (y no de sintaxis)?

	Respuesta	Explicación
❌	`if edad >= 18` (sin los dos puntos)	Error de **sintaxis**: el `if` debe terminar con `:`.
❌	`print("hola"` (sin el paréntesis de cierre)	Error de **sintaxis**: falta cerrar el paréntesis.
❌	`def calcular promedio(lista):`	Error de **sintaxis**: el nombre de una función no puede tener espacios.
✅	**`promedio = suma * cantidad`**	Python lo ejecuta sin quejarse, pero el resultado es incorrecto: para sacar el promedio hay que **dividir**, no multiplicar.

Sección 5: Relacionar columnas — Big O

Los números y las letras cambiaron en cada versión, así que aquí usamos directamente cada complejidad. Recuerda que de más rápida a más lenta van así: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$.

Acceder a un elemento por su índice

	Respuesta	Explicación
❌	O(log n)	No va descartando mitades: va directo al elemento.
❌	O(n)	No recorre la lista.
❌	O(n log n)	No ordena nada.
❌	$O(n^2)$	No compara elementos entre sí.
❌	$O(n^3)$	No hay ciclos anidados.
❌	$O(2^n)$	El trabajo no se duplica.
✅	**O(1)**	Python calcula directamente dónde está el elemento: tarda lo mismo con 10 elementos que con un millón.

Búsqueda binaria

	Respuesta	Explicación
❌	O(1)	No encuentra el dato en un solo paso.
❌	O(n)	Esa es la búsqueda **lineal**: revisar uno por uno.
❌	O(n log n)	Eso tarda **ordenar** con Merge Sort, no buscar.
❌	$O(n^2)$	No hay ciclos anidados.
❌	$O(n^3)$	No hay ciclos anidados.
❌	$O(2^n)$	El trabajo no se duplica: se reduce a la mitad.
✅	**O(log n)**	En cada paso descarta la mitad de la lista.

Encontrar el mayor de una lista

	Respuesta	Explicación
❌	O(1)	Hay que ver todos los elementos: el mayor puede estar al final.
❌	O(log n)	En una lista desordenada no puedes descartar la mitad sin revisarla.
❌	O(n log n)	No hace falta ordenar la lista.
❌	$O(n^2)$	No se compara todos contra todos: cada elemento solo se compara con “el mayor hasta ahora”.
❌	$O(n^3)$	No hay ciclos anidados.
❌	$O(2^n)$	El trabajo no se duplica.
✅	**O(n)**	Un solo recorrido de la lista.

Merge Sort

	Respuesta	Explicación
❌	O(1)	Ordenar no puede tardar lo mismo sin importar el tamaño.
❌	O(log n)	Solo dividir ya da log n niveles, y en cada nivel hay que mezclar todos los elementos.

	Respuesta	Explicación
❌	O(n)	Un solo recorrido no alcanza para ordenar comparando.
❌	O(n²)	Esa es la de **Bubble Sort**.
❌	O(n³)	No hay tres ciclos anidados.
❌	O(2ⁿ)	El trabajo no se duplica con cada elemento nuevo.
✅	**O(n log n)**	Hay log n niveles de división, y en cada nivel se mezclan los n elementos.

Bubble Sort

	Respuesta	Explicación
❌	O(1)	Ordenar no puede tardar lo mismo sin importar el tamaño.
❌	O(log n)	No descarta mitades.
❌	O(n)	Una sola pasada no basta: necesita varias.
❌	O(n log n)	Esa es la de **Merge Sort**.
❌	O(n³)	Son dos ciclos anidados, no tres.
❌	O(2ⁿ)	El trabajo no se duplica con cada elemento nuevo.
✅	**O(n²)**	Hace hasta n pasadas, y cada pasada hace hasta n comparaciones de vecinos.

Simplificar `2n² + 7n + 10`

	Respuesta	Explicación
❌	O(1)	El 10 es la constante, pero con n grande quien manda es n².
❌	O(log n)	No aparece ningún logaritmo.
❌	O(n)	7n crece más lento que n²; se descarta.
❌	O(n log n)	No aparece ningún logaritmo.
❌	O(n³)	El exponente más grande es 2, no 3.
❌	O(2ⁿ)	No hay ninguna n como exponente.
✅	**O(n²)**	Se queda el término que crece más rápido (2n²) y se quita la constante que lo multiplica (2).

Simplificar `n log n + n`

	Respuesta	Explicación
❌	O(1)	Sí depende de n.
❌	O(log n)	n log n crece mucho más rápido que log n.
❌	O(n)	n log n crece más rápido que n, así que es el término que manda.
❌	O(n²)	n log n crece más lento que n²: no se puede “redondear” hacia arriba.
❌	O(n³)	No hay n³.
❌	O(2ⁿ)	No hay ninguna n como exponente.

	Respuesta	Explicación
✅	**O(n log n)**	Se queda el término que crece más rápido.

Simplificar 5

	Respuesta	Explicación
❌	O(log n)	El 5 no depende de n.
❌	O(n)	Sería así si fuera 5n; el 5 solo no depende de n.
❌	O(n log n)	No depende de n.
❌	O(n^2)	No depende de n.
❌	O(n^3)	No depende de n.
❌	O(2^n)	No depende de n.
✅	**O(1)**	Cualquier constante se simplifica a O(1).

O(2^n) y O(n^3) **no** se usaban en ninguna.

Sección 6: Ciclo de vida de un programa

Las etapas: **1** Definición del problema · **2** Análisis · **3** Diseño de la solución · **4** Codificación · **5** Pruebas y depuración · **6** Documentación · **7** Liberación · **8** Mantenimiento. Los números de las etapas eran los mismos en todas las versiones.

Corregir un error que reportó un usuario *después* de publicar la app.

	Respuesta	Explicación
❌	1 Definición	El problema ya estaba definido desde antes.
❌	2 Análisis	No se están decidiendo entradas, salidas ni restricciones.
❌	3 Diseño	No se diseña el algoritmo desde cero.
❌	4 Codificación	Se toca código, pero de un programa que ya está publicado.
❌	5 Pruebas	Sería pruebas si el error se encontrara **antes** de publicar.
❌	6 Documentación	No se está explicando el código.
❌	7 Liberación	La app ya se había publicado.
✅	**8 Mantenimiento**	Todo lo que se corrige o cambia **después** de publicar es mantenimiento.

Corregir un error que encontraste al probar, *antes* de publicar la app.

	Respuesta	Explicación
❌	1 Definición	El problema ya estaba definido.
❌	2 Análisis	No se están decidiendo entradas ni salidas.
❌	3 Diseño	El algoritmo ya existía.
❌	4 Codificación	El código ya estaba escrito; ahora se está **probando**.

	Respuesta	Explicación
❌	6 Documentación	No se está explicando el código.
❌	7 Liberación	Todavía no se publica.
❌	8 Mantenimiento	Sería mantenimiento si la app ya estuviera publicada.
✅	**5 Pruebas y depuración**	Encontrar y corregir errores **antes** de publicar es depurar.

Decidir que el programa recibe calificaciones, regresa un promedio y que las calificaciones deben estar entre 0 y 10.

	Respuesta	Explicación
❌	1 Definición	La definición dice **qué problema** se resuelve y su alcance; aquí ya se detallan entradas, salidas y restricciones.
❌	3 Diseño	Todavía no se escriben los pasos del algoritmo.
❌	4 Codificación	No se escribe código.
❌	5 Pruebas	No se prueba nada.
❌	6 Documentación	No se explica código existente.
❌	7 Liberación	No se publica nada.
❌	8 Mantenimiento	El programa todavía no existe.
✅	**2 Análisis**	Entradas (calificaciones), salida (promedio) y restricciones (entre 0 y 10).

Escribir: INICIO, recibir calificaciones, sumar, dividir entre la cantidad, mostrar promedio, FIN.

	Respuesta	Explicación
❌	1 Definición	Ya no se define el problema: se escribe cómo resolverlo.
❌	2 Análisis	Las entradas y salidas ya se decidieron; aquí se escriben los pasos.
❌	4 Codificación	Todavía no está en Python: es pseudocódigo.
❌	5 Pruebas	No se prueba nada.
❌	6 Documentación	No explica un código existente: es el plan para escribirlo.
❌	7 Liberación	No se publica nada.
❌	8 Mantenimiento	El programa todavía no existe.
✅	**3 Diseño de la solución**	Escribir el algoritmo en pseudocódigo es diseñar la solución.

Agregar comentarios con `#` que expliquen qué hace la función `calcular_promedio`.

	Respuesta	Explicación
❌	1 Definición	El problema ya está definido.
❌	2 Análisis	No se deciden entradas ni salidas.
❌	3 Diseño	No se diseña el algoritmo.

	Respuesta	Explicación
❌	4 Codificación	Los comentarios no cambian lo que hace el programa.
❌	5 Pruebas	No se prueba nada.
❌	7 Liberación	No se publica nada.
❌	8 Mantenimiento	No se corrige ni se actualiza nada.
✅	**6 Documentación**	Explicar qué hace el código es documentarlo.

Subir la aplicación web a un servidor para que la usen los alumnos.

	Respuesta	Explicación
❌	1 Definición	El programa ya está terminado.
❌	2 Análisis	No se deciden entradas ni salidas.
❌	3 Diseño	No se diseña nada.
❌	4 Codificación	No se escribe código.
❌	5 Pruebas	No se prueba nada.
❌	6 Documentación	No se explica nada.
❌	8 Mantenimiento	El mantenimiento empieza **después** de publicar; aquí se está publicando.
✅	**7 Liberación (deploy)**	Ponerlo a disposición de los usuarios es liberarlo.

Traducir a Python el pseudocódigo (el de “INICIO, recibir calificaciones…”).

	Respuesta	Explicación
❌	1 Definición	El problema ya estaba definido.
❌	2 Análisis	Las entradas y salidas ya se decidieron.
❌	3 Diseño	El diseño es el pseudocódigo, que ya existía; ahora se **traduce**.
❌	5 Pruebas	Primero hay que tener el código para poder probarlo.
❌	6 Documentación	No se explica el código: se escribe.
❌	7 Liberación	No se publica nada.
❌	8 Mantenimiento	El programa todavía no existe.
✅	**4 Codificación**	Escribir el algoritmo en un lenguaje de programación es codificar.

Expresar con claridad el alcance del programa.

	Respuesta	Explicación
❌	2 Análisis	El análisis detalla entradas, salidas y restricciones; el **alcance** se fija antes.
❌	3 Diseño	Todavía no se escriben los pasos.
❌	4 Codificación	No se escribe código.
❌	5 Pruebas	No hay nada que probar.

	Respuesta	Explicación
❌	6 Documentación	No se explica un código existente.
❌	7 Liberación	No se publica nada.
❌	8 Mantenimiento	El programa todavía no existe.
✅	**1 Definición del problema**	Decir qué se va a resolver y hasta dónde llega es definir el problema.

Actualizar el programa porque cambió una de sus dependencias.

	Respuesta	Explicación
❌	1 Definición	El problema no cambió.
❌	2 Análisis	Las entradas y salidas no cambiaron.
❌	3 Diseño	El algoritmo no cambió.
❌	4 Codificación	Se toca código, pero de un programa que ya existe y está en uso.
❌	5 Pruebas	No se corrige un error encontrado al probar antes de publicar.
❌	6 Documentación	No se explica nada.
❌	7 Liberación	El programa ya estaba publicado.
✅	**8 Mantenimiento**	Adaptar el programa a cambios externos después de publicarlo es mantenimiento.

Revisar qué pasa si el programa recibe una lista vacía.

	Respuesta	Explicación
❌	1 Definición	No se define qué problema se resuelve.
❌	2 Análisis	No se deciden entradas ni salidas: se **revisa** cómo se comporta el programa.
❌	3 Diseño	No se escriben los pasos del algoritmo.
❌	4 Codificación	No se escribe código nuevo.
❌	6 Documentación	No se explica nada.
❌	7 Liberación	No se publica nada.
❌	8 Mantenimiento	No hay ningún error reportado después de publicar.
✅	**5 Pruebas y depuración**	Probar casos especiales (edge cases) como la lista vacía es parte de las pruebas.

Sección 7: Análisis de código

Programa A — años mundialistas

```
def busca_anio_mundialista_mexico(lista):

    for anio in lista:                              # n vueltas (una por año)

        lista_mundiales_mexico = [1970, 1986, 2026]
        for anio_mundial in lista_mundiales_mexico:  # siempre 3 vueltas
            print("Comparando ", anio, " con el mundial de México: ", anio_mundial)

            if anio == anio_mundial:
                print("El año ", anio, "es un año de mundial en Mexico.")

lista_anios = [2025, 2026, 2027]
busca_anio_mundialista_mexico(lista_anios)
```

Si `n` es la cantidad de años de `lista`, ¿cuál es la complejidad?

	Respuesta	Explicación
❌	O(1)	El ciclo de afuera da una vuelta por cada año: con más años, hay más trabajo.
❌	O(n²), porque con 3 años se hicieron 3 × 3 = 9 comparaciones.	**Es la trampa.** El 3 × 3 sale porque la lista de prueba tenía 3 años, pero el ciclo de adentro **siempre** da 3 vueltas (los 3 mundiales), sin importar `n`. Con 1,000 años serían 3,000 comparaciones, no 1,000,000.
❌	O(log n)	No se descarta ninguna mitad.
✅	**O(n)**	Son 3 × n comparaciones, y el 3 es una constante: 3n se simplifica a O(n). Sería O(n²) solo si el ciclo de adentro también dependiera de `n`.

Programa B — el banco que duplica tu dinero

```
def calcular_hace_cuantos_anios_comencamos_con_un_peso(cantidad_actual):
    anios = 0

    while cantidad_actual > 1:
        cantidad_actual = cantidad_actual / 2       # el valor se reduce a la mitad
        anios = anios + 1

    return anios

print(calcular_hace_cuantos_anios_comencamos_con_un_peso(64))   # Imprime 6
```

Si `n` es el valor de `cantidad_actual`, ¿cuál es la complejidad?

	Respuesta	Explicación
❌	O(1)	Con 64 da 6 vueltas y con 1,024 da 10: sí depende de `n`.
❌	O(n)	Con 64 serían 64 vueltas, pero solo da 6.
❌	O(n / 2), porque en cada vuelta se divide entre 2.	**Es la trampa.** Lo que se divide entre 2 es **el valor**, no el número de vueltas. Además, O(n/2) se simplificaría a O(n).
✅	**O(log n)**	El valor se reduce a la mitad en cada vuelta: 64 → 32 → 16 → 8 → 4 → 2 → 1 son 6 vueltas, y $\log_2 64 = 6$.

Programa C — ¿es número primo?

Completar las 5 líneas en blanco. Cada opción del banco se probó en Python con los números del −3 al 59. En cada línea, **solo una** opción hace que el programa funcione.

El programa completo, con las respuestas en las líneas que estaban en blanco:

```
def es_numero_primo(numero):
    if numero < 2:              # Línea 1: <
        return False            # Línea 2: False

    i = 2                       # Línea 3: 2
    while i < numero:
        print("Probando divisor: ", i)
        if numero % i == 0:     # Línea 4: %
            return False
        i = i + 1               # Línea 5: i + 1

    return True
```

Línea 1: `if numero ____ 2:`

	Opción	Explicación
❌	`False`, `True`, `1`, `2`, `i + 1`, `i - 1`, `numero + 1`	**Error de sintaxis**: entre `numero` y `2` tiene que ir un operador, no un valor.
❌	`%`	`numero % 2` es el residuo, y Python trata cualquier número distinto de 0 como verdadero. Todos los impares (3, 5, 7…) se declararían “no primos”.
❌	`/`	`numero / 2` solo es 0 cuando `numero` es 0. El 2, el 3, el 5… se declararían “no primos”.
❌	`//`	`numero // 2` es 0 solo con 0 y 1. Todos los números de 2 en adelante se declararían “no primos”.
❌	`>`	Haría al revés: los números mayores que 2 se declararían “no primos” y los negativos, “primos”.
❌	`<=`	Casi funciona, pero el **2** se declararía “no primo”, y el 2 sí es primo.
✅	**`<`**	Los números menores que 2 no son primos (por convención).

Línea 2: `return ____`

	Opción	Explicación
❌	`<`, `<=`, `>`, `%`, `/`, `//`	**Error de sintaxis**: un operador solo no es un valor que se pueda regresar.
❌	`i + 1`, `i - 1`	**Error**: `i` todavía no existe en esa línea (se crea más abajo).
❌	`True`	Diría que los números menores que 2 sí son primos.
❌	`1`, `2`, `numero + 1`	Regresarían un **número** en lugar de `False`, y la función debe regresar `True` o `False`.
✅	**`False`**	Los números menores que 2 **no** son primos.

Línea 3: `i = ____`

	Opción	Explicación
❌	`<`, `<=`, `>`, `%`, `/`, `//`	**Error de sintaxis**: `i` necesita un valor, no un operador.
❌	`i + 1`, `i - 1`	**Error**: `i` todavía no existe; justo se está creando en esta línea.
❌	`False`	`False` vale 0, y `numero % 0` es una **división entre cero** (`ZeroDivisionError`).
❌	`1`, `True`	`True` vale 1. Todo número es divisible entre 1, así que **ningún** número sería primo.
❌	`numero + 1`	`i` empezaría mayor que `numero` y el ciclo nunca entraría: todos los números serían "primos" (4, 6, 8…).
✅	**`2`**	El 1 no sirve como divisor de prueba, así que el primer divisor que hay que probar es el 2.

Línea 4: `if numero ____ i == 0:`

	Opción	Explicación
❌	`False`, `True`, `1`, `2`, `i + 1`, `i - 1`, `numero + 1`	**Error de sintaxis**: entre `numero` e `i` tiene que ir un operador.
❌	`/`	`numero / i` nunca da 0 dentro del ciclo: nunca detecta un divisor y todos los números serían "primos" (4, 6, 8…).
❌	`//`	`numero // i` solo da 0 si `i` es mayor que `numero`, y eso nunca pasa dentro del ciclo. El mismo problema que con `/`.
❌	`<`, `<=`, `>`	Python lo leería como dos comparaciones (por ejemplo, `numero < i` **y** `i == 0`); como `i` nunca es 0, todos los números serían "primos".
✅	**`%`**	`numero % i == 0` significa "el residuo es 0", es decir, que `i` divide exactamente a `numero`.

Línea 5: `i = ____`

	Opción	Explicación
❌	`<`, `<=`, `>`, `%`, `/`, `//`	**Error de sintaxis**: `i` necesita un valor, no un operador.
❌	`False`	`i` valdría 0 y en la siguiente vuelta habría una **división entre cero**.
❌	`2`	`i` se quedaría siempre en 2: con un número impar, el ciclo **nunca termina**.
❌	`1`, `True`, `i - 1`	En la segunda vuelta se probaría el **1**, que divide a todos: los primos impares (3, 5, 7…) se declararían "no primos".
❌	`numero + 1`	Solo probaría el 2 y saldría del ciclo: el 9, el 15, el 21… se declararían "primos".
✅	**`i + 1`**	Avanza al siguiente divisor: 2, 3, 4, 5…

¿Cuántas veces se imprime `Probando divisor:` al ejecutar `es_numero_primo(19)`?

	Respuesta	Explicación
❌	1	Solo pasaría si el primer divisor probado (el 2) dividiera a 19 y la función se detuviera ahí.
❌	18	Sería contar del 1 al 18, pero `i` empieza en **2**.
❌	19	Sería contar del 1 al 19, pero `i` empieza en 2 y el ciclo se detiene **antes** de llegar a 19 (`i < numero`).
✅	**17**	Prueba los divisores del 2 al 18 (son 17), y ninguno divide a 19.

Si `n` es el valor de `numero`, ¿cuál es la complejidad en el peor caso?

	Respuesta	Explicación
❌	O(1)	Con un primo grande, el ciclo da muchas vueltas: sí depende de `n`.
❌	O(log n)	`i` avanza de 1 en 1; no se divide nada a la mitad.
❌	O(n²)	Hay un solo ciclo, no dos ciclos anidados.
✅	**O(n)**	El peor caso es cuando el número es primo: el ciclo da n − 2 vueltas, que se simplifica a O(n).

Sección 8: Sigue el algoritmo

¿Cuánto vale R al terminar? (avanzar = izquierda, regresar = derecha)

	Respuesta	Explicación
❌	3	Sale si te mueves en la dirección equivocada o cuentas mal las posiciones.
❌	9	Igual: sale de un error al moverte o en las cuentas.
❌	Depende del número de cuenta de cada quien.	Parece que depende, pero los dígitos se **cancelan** (ver la tabla de abajo).
✅	**5**	Da 5 con **cualquier** número de cuenta.

Por qué siempre da 5. Llamemos **x** al 2.º dígito desde la derecha y **y** al 3.º:

Paso	Dónde quedas (contando desde la derecha)	R
0	—	0
1	2.º dígito (x)	2x
2	Avanzas a la 4.ª y regresas a la 3.ª (y)	2x + y
3	6.º dígito	2x + y + 10
4	3.º dígito otra vez (y)	2x + 10
5	2.º dígito otra vez (x)	(2x + 10) ÷ 2 − x = x + 5 − x = **5**

El paso 4 resta el mismo dígito que sumó el paso 2, y el paso 5 resta la mitad de lo que sumó el paso 1: todo se cancela y solo queda la mitad del 10.

Ejemplo con 303312329:

Paso	Dígito donde quedas	R
1	2	2 × 2 = 4
2	3	4 + 3 = 7
3	3	7 + 10 = 17
4	3	17 − 3 = 14
5	2	14 ÷ 2 − 2 = **5**

Pregunta de rescate

Hay 10 tipos de personas en el mundo: las que entienden binario y **las que no**.

El chiste: **10** en binario vale **2** (1 × 2 + 0 × 1). Por eso la frase dice “10 tipos” y luego nombra solo dos grupos. Se aceptan respuestas con el mismo sentido, como “las que no lo entienden”.